Date : _____

OOP

Object Oriented Programming

Date :

* Procedure Programming

- This type of program consists of collection of procedures / modules / fn.
- It is nothing but the part of whole program which is performing some particular task.
- eg. C, FORTRAN, assembly prog., etc.
- Main focus on fn, not on data

- Limitations

- Security
- Data hiding
- Scalability
- less maintainability.

* Object oriented Programming.

- OOP is new approach to modern Programming.

Date :

- eg. C++, Java, PHP, etc

- It uses diff^n object oriented concepts like obj, classes, polymorphism, encapsulat^n, inheritance, data binding, etc

• Need.

- It provides obj, closes --- data binding, etc

- It is very easy to develop.

- Software code reusability is possible.

- Complexity can be easily managed.

* Procedure OP. OOP.

(i) It depends upon fn (i) obj.

(ii) Main focus on fn (ii) Main focus on ~~obj~~ data

(iii) Less secure (iii) More Secure

(iv) It uses top down approach (iv) bottom up approach.

* Features of OOP.

• Namespaces

- It is declarative region that provides a scope to the identifiers (the name of types, f^n, variables, etc.) inside it.

• Classes.

- The building block of C++ that leads to OOP is a class.

- It is a user defined data type, which holds its own data members & member f^n, which can be accessed & used by creating an instance of that class.

Date :

• Objects.

- An object is an instance of a class.

- An obj. is an identifiable entity with some characteristics & behaviours.

• Data Members.

- The variables which are declared in any class by using any fundamental data types (int, char, float) or derived data types (class, structure) are known as D.M.

- Private & Public Members.

• Methods.

- Method is procedure or fn in OOP Concepts.

- It works as fn..

Date :

- It defined inside a class.

- It can be private, public or protected.

• Messages.

- Objs. communicate with one another by sending & receiving info. to each other.

• Data Encapsulation.

- It defined as wrapping up of data & info. under a single unit.

- It also defined as binding together the data & the fn that manipulate them.

• Data abstraction

- It refers to using very essential features without adding the background

Date :

details of fn.

- It is way info. hiding from outside the world.

• Inheritance

- The capability of class to derive properties & characteristics from another class.

• Polymorphism.

- It means having many forms

- The ability of msg to be displayed in more than one form.

• Benefits of OOP.

- Quality & Productivity of S/w is increased
- Code reuse.
- Complexity easily managed.
- data hiding

Date :

* Access Specifiers & Modifiers.

① Public :
- Public variables or fn can be used by everyone.
- It can be accessed by member fn of class as well as from other class.

② Private :
- Private members can be accessed only using the member fn of the class.

③ Protected :
- Protected members of the class are accessible by class fn.
- They are also accessible ~~by~~ to subclass of that class.

* New Operator in class & obj. (tut 26)

- The memory for the objected is

Date :

is allocated using operator new from heap.
- The constructor of the class is invoked to properly initialize this memory.

* Nesting of Member function (tut 27)

- A member fn can call another member fn of the same class for that you do not need an object.

* Static Data Members (tut 28).

- When a static data member is created, there is only a single copy of the data member which is shared betn all obj. of class

- Static variables have allocated static memory.

Date :

* Static Member fn.

- Static member fn can access only static members.

- They are called using class name & not by obj. name.

- fn should be declared with static keyword.

* friend class & functions. (tut 29)

- A friend class can access private & protected members of other classes in which it is declared as a friend.

- It is sometimes useful to allow a particular class to access private & protected members of other class.

Date :

- Syntax.

class Geeks {

friend class GFG ; } - Base class
}

class GFG {
Statements ; } friend class
}

- Adv. :

- It provides additional functionality.
- It is used to create many ip/op fn
- It is used for operator overloading.
- It is used to create effective code

- Disadv :

- It doesn't have storage class specifier.
- A derived class can't inherit friend fn.

Date :

* Constructors. (tut 30).

- It is used to initialize the objects.

- It is special fn which is automatically get called during object creation.

- It is same as class name.

- It do not return value & return type.

* Types of Constructors.

① Parameterized Constructor

- Parameterized constructor is used to pass the arg. to constructor during obj. creation.

- Passing values can be used for initialization purposes.

Date :

2. Multiple Constructors within class.

- We can have multiple constructors in class also.

- It means class can have combinat" of default, parameterized, etc constructor

3. Default Constructors

- If we are not passing the value to during obj. creation then default values are used.

4. Copy Constructors.

- It is used to initialize one obj. from another obj. of same type

5. Default Copy Constructors

- C++ create default constructor if

Date :

we don't define it.

- Constructor created by compiler has has empty body.

- It does not assign default values to data members.

* Destructors (tut 33)

- It is used to destroy the objects in C++.

- They destroy the obj. which are created by constructor

- (~) tilde sign used

* Constructor Overloading (tut 31)
* Constructor with default arg. (tut 32)
* Dynamic allocation (new) (tut 33)
* Dynamic deallocation (delete) (tut 34)

Date :

* Inheritance (C++)

- Single Inheritance (tut 36)
- Multiple ⊥ (tut 37)
- Multilevel ⊥ (tut 38)
- Hierarchical ⊥ (tut 39)
- Hybrid ⊥ (tut 40).

* Access Modifiers (C++).

```
class Parent {
  public:
    int x;
  protected:
    int y;
  private:
    int z;
};
class child1 : public Parent {
  // x will remain public
  // y ——————— protected
  // z will not be accessible
};
```

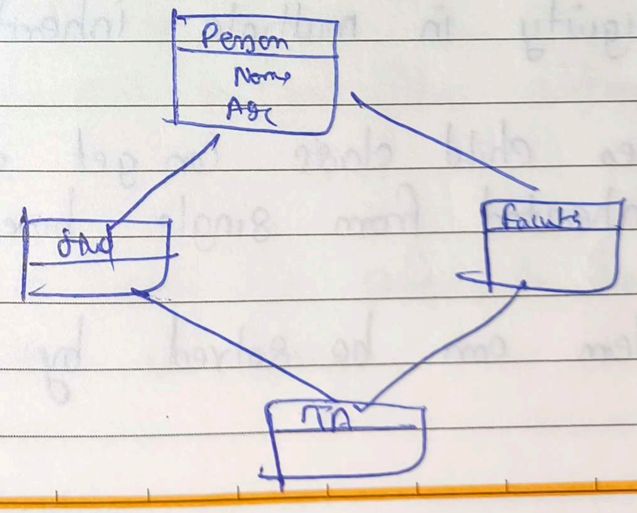
Date :

```
class child2 : private Parent {
  // x will be private
  // y ——————— private
  // z will be inaccessible
};

class child3 : protected Parent {
  // x will protected
  // y ——————— protected
  // z will be inaccessible
};
```

* Diamond Problem :

- It occurs when two superclass of a class have a common base class.



Date :

* Ambiguity in Multiple Inheritance.

- Ambiguity : It can result if base & derived classes have members with the same names.

- This situations arises when multiple base class have members with same name.

- Ambiguity resolved using scope resolved operations.

* Virtual Base class.

- Virtual bases class is used to remove the ambiguity in multiple inheritance.

- Sometimes child class can get duplicate member inherited from single base class.

- This problem can be solved by virtual base class.

Date :

* Pointer : New & delete keyword

- New is used to allocate memory
- delete is used to deallocate the memory.
- It is more like malloc() & free() in C.

* Pointers to objects.

- Pointer is used to access elements of obj.
- Obj. ptr. are useful in creating obj. at run time.

* this pointer.

- It is used to get the address of own obj.

- It is used to refer to the current instance of class

Date :

* Function Pointers

- It is used in dynamic binding
- It also known as call back fn
- It can be used to refer to functions

* Null pointer

- A pointer that is assigned NULL is called null ptr.

* Void Pointer

- A Void ptr is ptr that has no associated data type with it.
- A void ptr can hold address of any type & can be typecasted to any type.

Date :

* Encapsulations :

- It defined as the wrapping up of data & info. in a single unit. (class)

- Imp Properties
  - Data Protection
  - Info. hiding

* Polymorphism (CT)

- Compile time
- Run time
- fn overloading
- Operator
- fn overriding.

* friend function. (Previous)

* Type Casting (Polymorphism)

- Type casting is converting one data type into another one.

Date :

* Implicit Type Casting

- It means conversion of data type without loosing its original meaning.

* Explicit Type Casting

- In implicit type convenion, the data type is converted automatically
- There are some scenarios in which we may have to force type convenion

* Pure Virtual Fn

- It is type of VF which is not defined in base class.
- Only derive class can define fn

* Abstract Base Class

- They are classes that can only be used as base classes, & thus are allowed to have virtual member fn without definition

Date :

- Pillars of OOP
  (i) Inheritance
  (ii) Polymorphism
  (iii) Encapsulation
  (iv) Data Abstraction.

Basic OOPs Interview Questions

1. What is the need for OOPs?

There are many reasons why OOPs is mostly preferred, but the most important among them are:

- OOPs helps users to understand the software easily, although they don't know the actual implementation.
- With OOPs, the readability, understandability, and maintainability of the code increase multifold.
- Even very big software can be easily written and managed easily using OOPs.

2. What are some major Object Oriented Programming languages?

The programming languages that use and follow the Object-Oriented Programming paradigm or OOPs, are known as Object-Oriented Programming languages. Some of the major Object-Oriented Programming languages include:

- **Java**
- **C++**
- **Javascript**
- **Python**
- **PHP**

And many more.

3. What are some other programming paradigms other than OOPs?

Programming paradigms refers to the method of classification of programming languages based on their features. There are mainly two types of Programming Paradigms:

- Imperative Programming Paradigm
- Declarative Prog

Now, these paradigms can be further classified based:

1. Imperative Programming Paradigm: Imperative programming focuses on HOW to execute program logic and defines control flow as statements that change a program state. This can be further classified as:

- PHP

And many more.

3. What are some other programming paradigms other than OOPs?

Programming paradigms refers to the method of classification of programming languages based on their features. There are mainly two types of Programming Paradigms:

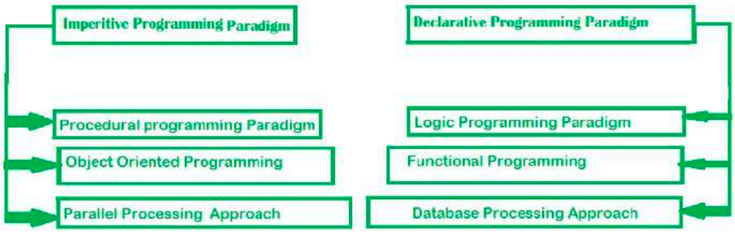
- Imperative Programming Paradigm
- Declarative Programming Paradigm

Now, these paradigms can be further classified based:

1. Imperative Programming Paradigm: Imperative programming focuses on HOW to execute program logic and defines control flow as statements that change a program state. This can be further classified as:
a) Procedural Programming Paradigm: Procedural programming specifies the steps a program must take to reach the desired state, usually read in order from top to bottom.
b) Object-Oriented Programming or OOP: Object-oriented programming (OOP) organizes programs as objects, that contain some data and have some behavior.
c) Parallel Programming: Parallel programming paradigm breaks a task into subtasks and focuses on executing them simultaneously at the same time.

2. Declarative Programming Paradigm: Declarative programming focuses on WHAT to execute and defines program logic, but not a detailed control flow. Declarative paradigm can be further classified into:
a) Logical Programming Paradigm: Logical programming paradigm is based on formal logic, which refers to a set of sentences expressing facts and rules about how to solve a problem
b) Functional Programming Paradigm: Functional programming is a programming paradigm where programs are constructed by applying and composing functions.
c) Database Programming Paradigm: Database programming model is used to manage data and information structured as fields, records, and files.

Programming Pardigms



InterviewBit

You can download a PDF version of Oops Interview Questions. Download PDF

4. What is meant by Structured Programming?

Structured Programming refers to the method of programming which consists of a completely structured control flow. Here structure refers to a block, which contains a set of rules, and has a definitive control flow, such as (if/then/else), (while and for), block structures, and subroutines.

Nearly all programming paradigms include Structured programming, including the OOPs model.

5. What are the

4. What is meant by Structured Programming?

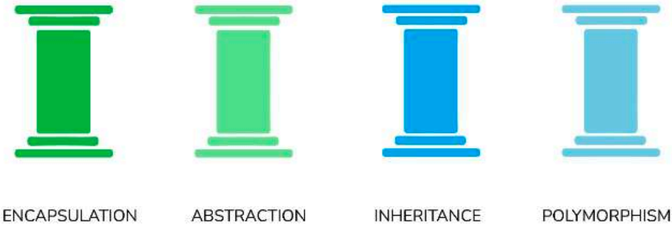
Structured Programming refers to the method of programming which consists of a completely structured control flow. Here structure refers to a block, which contains a set of rules, and has a definitive control flow, such as (if/then/else), (while and for), block structures, and subroutines.

Nearly all programming paradigms include Structured programming, including the OOPs model.

5. What are the main features of OOPs?

OOPs or Object Oriented Programming mainly comprises of the below four features, and make sure you don't miss any of these:

- Inheritance
- Encapsulation
- Polymorphism
- Data Abstraction



6. What are some advantages of using OOPs?

- OOPs is very helpful in solving very complex level of problems.
- Highly complex programs can be created, handled, and maintained easily using object-oriented programming.
- OOPs, promote code reuse, thereby reducing redundancy.
- OOPs also helps to hide the unnecessary details with the help of Data Abstraction.
- OOPs, are based on a bottom-up approach, unlike the Structural programming paradigm, which uses a top-down approach.

6. What are some advantages of using OOPs?

- OOPs is very helpful in solving very complex level of problems.
- Highly complex programs can be created, handled, and maintained easily using object-oriented programming.
- OOPs, promote code reuse, thereby reducing redundancy.
- OOPs also helps to hide the unnecessary details with the help of Data Abstraction.
- OOPs, are based on a bottom-up approach, unlike the Structural programming paradigm, which uses a top-down approach.
- Polymorphism offers a lot of flexibility in OOPs.

7. Why is OOPs so popular?

OOPs programming paradigm is considered as a better style of programming. Not only it helps in writing a complex piece of code easily, but it also allows users to handle and maintain them easily as well. Not only that, the main pillar of OOPs - Data Abstraction, Encapsulation, Inheritance, and Polymorphism, makes it easy for programmers to solve complex scenarios. As a result of these, OOPs is so popular.

8. What is meant by the term OOPs?

OOPs refers to Object-Oriented Programming. It is the programming paradigm that is defined using objects. Objects can be considered as real-world instances of entities like class, that have some characteristics and behaviors.

Advanced OOPs Interview Questions

1. What are access specifiers and what is their significance?

Access specifiers, as the name suggests, are a special type of keywords, which are used to control or specify the accessibility of entities like classes, methods, etc. Some of the access specifiers or access modifiers include “private”, “public”, etc. These access specifiers also play a very vital role in achieving Encapsulation - one of the major features of OOPs.

2. Are there any limitations of Inheritance?

Yes, with more powers comes more complications. Inheritance is a very powerful feature in OOPs, but it has som

some characteristics and behaviors.

Advanced OOPs Interview Questions

1. What are access specifiers and what is their significance?

Access specifiers, as the name suggests, are a special type of keywords, which are used to control or specify the accessibility of entities like classes, methods, etc. Some of the access specifiers or access modifiers include “private”, “public”, etc. These access specifiers also play a very vital role in achieving Encapsulation - one of the major features of OOPs.

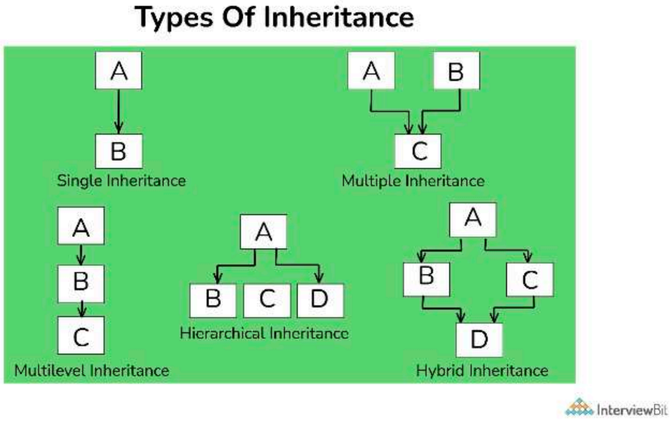
2. Are there any limitations of Inheritance?

Yes, with more powers comes more complications. Inheritance is a very powerful feature in OOPs, but it has some limitations too. Inheritance needs more time to process, as it needs to navigate through multiple classes for its implementation. Also, the classes involved in Inheritance - the base class and the child class, are very tightly coupled together. So if one needs to make some changes, they might need to do nested changes in both classes. Inheritance might be complex for implementation, as well. So if not correctly implemented, this might lead to unexpected errors or incorrect outputs.

3. What are the various types of inheritance?

The various types of inheritance include:

- Single inheritance
- Multiple inheritances
- Multi-level inheritance
- Hierarchical inheritance
- Hybrid inheritance



4. What is a subclass?

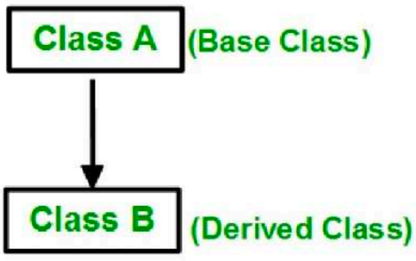
The subclass is a part of Inheritance. The subclass is an entity, which inherits from another class. It is also known as the child class.

5. Define a superclass?

Superclass is also a part of Inheritance. The superclass is an entity, which allows subclasses or child classes to inher

5. Define a superclass?

Superclass is also a part of Inheritance. The superclass is an entity, which allows subclasses or child classes to inherit from itself.



InterviewBit

6. What is an interface?

An interface refers to a special type of class, which contains methods, but not their definition. Only the declaration of methods is allowed inside an interface. To use an interface, you cannot create objects. Instead, you need to implement that interface and define the methods for their implementation.

7. What is meant by static polymorphism?

Static Polymorphism is commonly known as the Compile time polymorphism. Static polymorphism is the feature by which an object is linked with the respective function or operator based on the values during the compile time. Static or Compile time Polymorphism can be achieved through Method overloading or operator overloading.

8. What is meant by dynamic polymorphism?

Dynamic Polymorphism or Runtime polymorphism refers to the type of Polymorphism in OOPs, by which the actual implementation of the function is decided during the runtime or execution. The dynamic or runtime polymorphism can be achieved with the help of method overriding.

9. What is the difference between overloading and overriding?

Overloading is a compile-time polymorphism feature in which an entity has multiple implementations with the same name. For example, Method overloading and Operator overloading.

Whereas Overriding is a runtime polymorphism feature in which an entity has the same name, but its implementation changes during execution. For example, Method overriding.

Image

10. How is data abstraction accomplished?

Data abstraction is accomplished with the help of abstract methods or abstract classes.

11. What is an abstract class?

An abstract class is a

Image

10. How is data abstraction accomplished?

Data abstraction is accomplished with the help of abstract methods or abstract classes.

11. What is an abstract class?

An abstract class is a special class containing abstract methods. The significance of abstract class is that the abstract methods inside it are not implemented and only declared. So as a result, when a subclass inherits the abstract class and needs to use its abstract methods, they need to define and implement them.

12. How is an abstract class different from an interface?

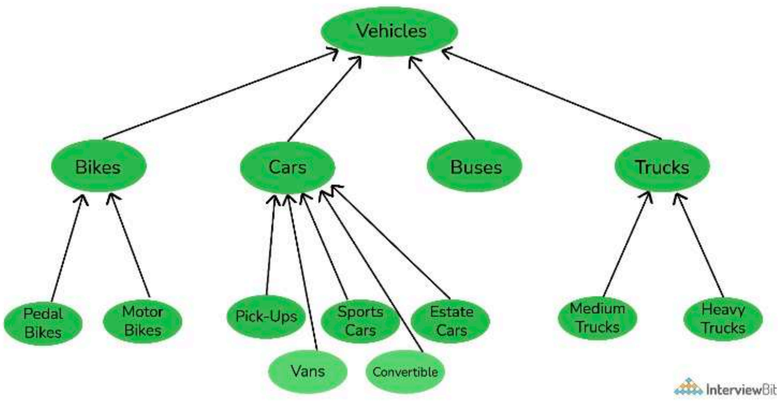
Interface and abstract classes both are special types of classes that contain only the methods declaration and not their implementation. But the interface is entirely different from an abstract class. The main difference between the two is that when an interface is implemented, the subclass must define all its methods and provide its implementation. Whereas in object-oriented programming, when a subclass inherits from an abstract class with abstract methods, the subclass is generally required to provide concrete implementations for all of those abstract methods in the abstract class unless the subclass itself is declared as abstract.

Also, an abstract class can contain abstract methods as well as non-abstract methods.

13. Explain Inheritance with an example?

Inheritance is one of the major features of object-oriented programming, by which an entity inherits some characteristics and behaviors of some other entity and makes them their own. Inheritance helps to improve and facilitate code reuse.

Let me explain to you with a common example. Let's take three different vehicles - a car, truck, or bus. These three are entirely different from one another with their own specific characteristics and behavior. But. in all three, you will find some common elements, like steering wheel, accelerator, clutch, brakes, etc. Though these elements are used in different vehicles, still they have their own features which are common among all vehicles. This is achieved with inheritance. The car, the truck, and the bus have all inherited the features like steering wheel, accelerator, clutch, brakes, etc, and used them as their own. Due to this, they did not have to create these components from scratch, thereby facilitating code reuse.



14. What is an exception?

An exception can be considered as a special event, which is raised during the execution of a program at runtime, that brings the execution to a halt. The reason for the exception is mainly due to a position in the program, where the user wants to do something for which the program is not specified, like undesirable input.

15. What is meant by exception handling?

No one wants its sof

is not specified, like undesirable input.

15. What is meant by exception handling?

No one wants its software to fail or crash. Exceptions are the major reason for software failure. The exceptions can be handled in the program beforehand and prevent the execution from stopping. This is known as exception handling.
So exception handling is the mechanism for identifying the undesirable states that the program can reach and specifying the desirable outcomes of such states.
Try-catch is the most common method used for handling exceptions in the program.

16. What is meant by Garbage Collection in OOPs world?

Object-oriented programming revolves around entities like objects. Each object consumes memory and there can be multiple objects of a class. So if these objects and their memories are not handled properly, then it might lead to certain memory-related errors and the system might fail.

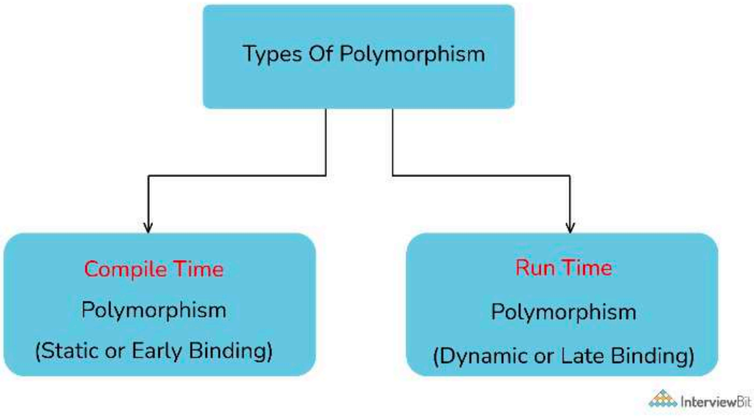
Garbage collection refers to this mechanism of handling the memory in the program. Through garbage collection, the unwanted memory is freed up by removing the objects that are no longer needed.

17. Can we run a Java application without implementing the OOPs concept?

No. Java applications are based on Object-oriented programming models or OOPs concept, and hence they cannot be implemented without it.

However, on the other hand, C++ can be implemented without OOPs, as it also supports the C-like structural programming model.

18. What is Compile time Polymorphism and how is it different from Runtime Polymorphism?



Compile Time Polymorphism: Compile time polymorphism, also known as Static Polymorphism, refers to the type of Polymorphism that happens at compile time. What it means is that the compiler decides what shape or value has to be taken by the entity in the picture.

Example:

```
// In this program, we will see how multiple functions are created with the same name,
// but the compiler decides which function to call easily at the compile time itself.
class CompileTimePolymorphism{
  // 1st method with name add
  public int add(int x, int y){
  return x+y;
  }
  // 2nd method with name add
  public int add(int x, int y, int z){
  return x+y+z;
  }
  // 3rd method with r
  public int add(doubl
```

Compile Time Polymorphism: Compile time polymorphism, also known as Static Polymorphism, refers to the type of Polymorphism that happens at compile time. What it means is that the compiler decides what shape or value has to be taken by the entity in the picture.

Example:

```
// In this program, we will see how multiple functions are created with the same name,
// but the compiler decides which function to call easily at the compile time itself.
class CompileTimePolymorphism{
  // 1st method with name add
  public int add(int x, int y){
  return x+y;
  }
  // 2nd method with name add
  public int add(int x, int y, int z){
  return x+y+z;
  }
  // 3rd method with name add
  public int add(double x, int y){
  return (int)x+y;
  }
  // 4th method with name add
  public int add(int x, double y){
  return x+(int)y;
  }
}
class Test{
  public static void main(String[] args){
  CompileTimePolymorphism demo=new CompileTimePolymorphism();
  // In the below statement, the Compiler looks at the argument types and decides to call method 1
  System.out.println(demo.add(2,3));
  // Similarly, in the below statement, the compiler calls method 2
  System.out.println(demo.add(2,3,4));
  // Similarly, in the below statement, the compiler calls method 4
  System.out.println(demo.add(2,3.4));
  // Similarly, in the below statement, the compiler calls method 3
  System.out.println(demo.add(2.5,3));
  }
}
```

In the above example, there are four versions of add methods. The first method takes two parameters while the second one takes three. For the third and fourth methods, there is a change of order of parameters. The compiler looks at the method signature and decides which method to invoke for a particular method call at compile time.

Runtime Polymorphism: Runtime polymorphism, also known as Dynamic Polymorphism, refers to the type of Polymorphism that happens at the run time. What it means is it can't be decided by the compiler. Therefore what shape or value has to be taken depends upon the execution. Hence the name Runtime Polymorphism.

Example:

```
class AnyVehicle{
  public void move(){
  System.out.println("Any vehicle should move!!");
  }
}
class Bike extends AnyVehicle{
  public void move(){
  System.out.println("Bike can move too!!");
  }
}
class Test{
  public static void main(String[] args){
  AnyVehicle vehicle = new Bike();
  // In the above statement, as you can see, the object vehicle is of type AnyVehicle
  // But the output of the below statement will be "Bike can move too!!",
  // because the actual implementation of object 'vehicle' is decided during runtime vehicle.move();
  vehicle = new AnyVe
  // Now, the output c
```

Hence the name Runtime Polymorphism.

Example:

```
class AnyVehicle{
  public void move(){
  System.out.println("Any vehicle should move!!");
  }
}
class Bike extends AnyVehicle{
  public void move(){
  System.out.println("Bike can move too!!");
  }
}
class Test{
  public static void main(String[] args){
  AnyVehicle vehicle = new Bike();
  // In the above statement, as you can see, the object vehicle is of type AnyVehicle
  // But the output of the below statement will be "Bike can move too!!",
  // because the actual implementation of object 'vehicle' is decided during runtime vehicle.move();
  vehicle = new AnyVehicle();
  // Now, the output of the below statement will be "Any vehicle should move!!",
  vehicle.move();
  }
}
```

As the method to call is determined at runtime, as shown in the above code, this is called runtime polymorphism.

19. What is a class?

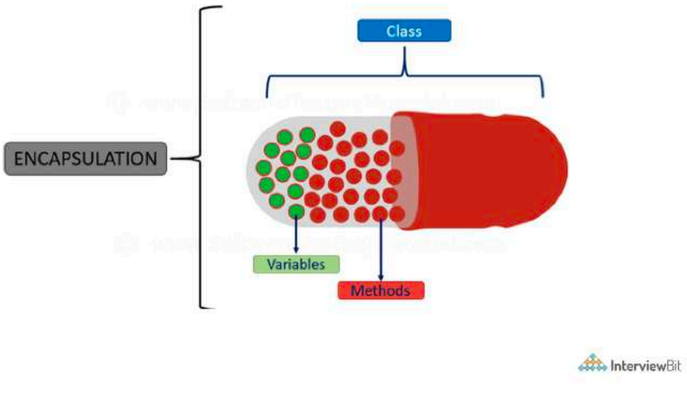
A class can be understood as a template or a blueprint, which contains some values, known as member data or member, and some set of rules, known as behaviors or functions. So when an object is created, it automatically takes the data and functions that are defined in the class. Therefore the class is basically a template or blueprint for objects. Also one can create as many objects as they want based on a class.

For example, first, a car's template is created. Then multiple units of car are created based on that template.

20. What is an object?

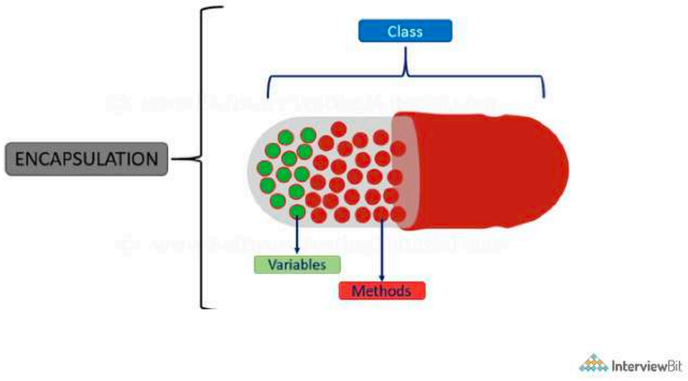
An object refers to the instance of the class, which contains the instance of the members and behaviors defined in the class template. In the real world, an object is an actual entity to which a user interacts, whereas class is just the blueprint for that object. So the objects consume space and have some characteristic behavior.
For example, a specific car.

21. What is encapsulation?



One can visualize Encapsulation as the method of putting everything that is required to do the job, inside a capsule and presenting that capsule to the user. What it means is that by Encapsulation, all the

21. What is encapsulation?



One can visualize Encapsulation as the method of putting everything that is required to do the job, inside a capsule and presenting that capsule to the user. What it means is that by Encapsulation, all the necessary data and methods are bind together and all the unnecessary details are hidden to the normal user. So Encapsulation is the process of binding data members and methods of a program together to do a specific job, without revealing unnecessary details.

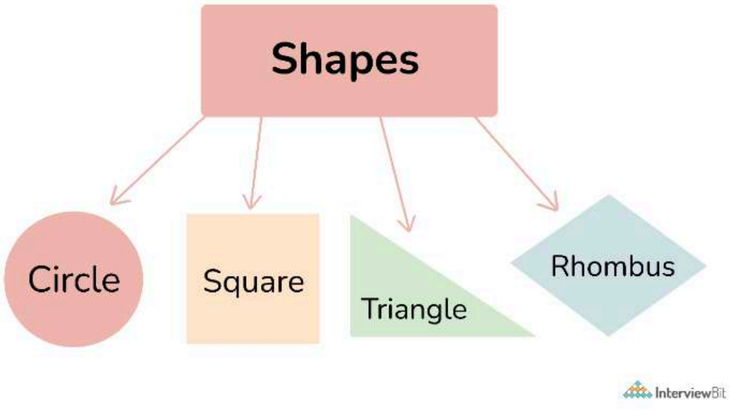
Encapsulation can also be defined in two different ways:

1) **Data hiding:** Encapsulation is the process of hiding unwanted information, such as restricting access to any member of an object.

2) **Data binding:** Encapsulation is the process of binding the data members and the methods together as a whole, as a class.

22. What is Polymorphism?

Polymorphism is composed of two words - “poly” which means “many”, and “morph” which means “shapes”. Therefore Polymorphism refers to something that has many shapes.



In OOPs, Polymorphism refers to the process by which some code, data, method, or object behaves differently under different circumstances or contexts. Compile-time polymorphism and Run time polymorphism are the two types of polymorphisms in OOPs languages.

23. How does C++ support Polymorphism?

C++ is an Object-oriented programming language and it supports Polymorphism as well:

- Compile Time Polymorphism: C++ supports compile-time polymorphism with the help of features like templates, function overloading, and default arguments.
- Runtime Polymorphism: C++ supports Runtime polymorphism with the help of features like virtual functions object in referen

Run time polymorphism are the two types of polymorphisms in OOPs languages.

23. How does C++ support Polymorphism?

C++ is an Object-oriented programming language and it supports Polymorphism as well:

- Compile Time Polymorphism: C++ supports compile-time polymorphism with the help of features like templates, function overloading, and default arguments.
- Runtime Polymorphism: C++ supports Runtime polymorphism with the help of features like virtual functions. Virtual functions take the shape of the functions based on the type of object in reference and are resolved at runtime.

24. What is meant by Inheritance?

The term “inheritance” means “receiving some quality or behavior from a parent to an offspring.” In object-oriented programming, inheritance is the mechanism by which an object or class (referred to as a child) is created using the definition of another object or class (referred to as a parent). Inheritance not only helps to keep the implementation simpler but also helps to facilitate code reuse.

25. What is Abstraction?

If you are a user, and you have a problem statement, you don't want to know how the components of the software work, or how it's made. You only want to know how the software solves your problem. Abstraction is the method of hiding unnecessary details from the necessary ones. It is one of the main features of OOPs.
For example, consider a car. You only need to know how to run a car, and not how the wires are connected inside it. This is obtained using Abstraction.

26. How much memory does a class occupy?

Classes do not consume any memory. They are just a blueprint based on which objects are created. Now when objects are created, they actually initialize the class members and methods and therefore consume memory.

27. Is it always necessary to create objects from class?

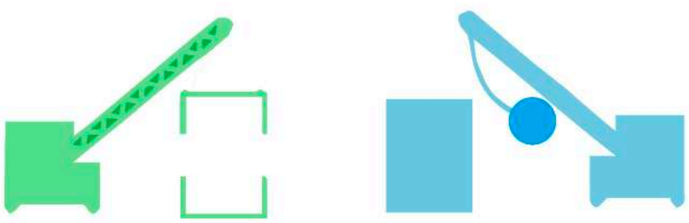
No. An object is necessary to be created if the base class has non-static methods. But if the class has static methods, then objects don’t need to be created. You can call the class method directly in this case, using the class name.

28. What is a constructor?

Constructors are special methods whose name is the same as the class name. The constructors serve the special purpose of initializing the objects.
For example, suppose there is a class with the name “MyClass”, then when you instantiate this class, you pass the syntax:

```
MyClass myClassObject = new MyClass();
```

Now here, the method called after “new” keyword - MyClass(), is the constructor of this class. This will help to instantiate the member data and methods and assign them to the object myClassObject.



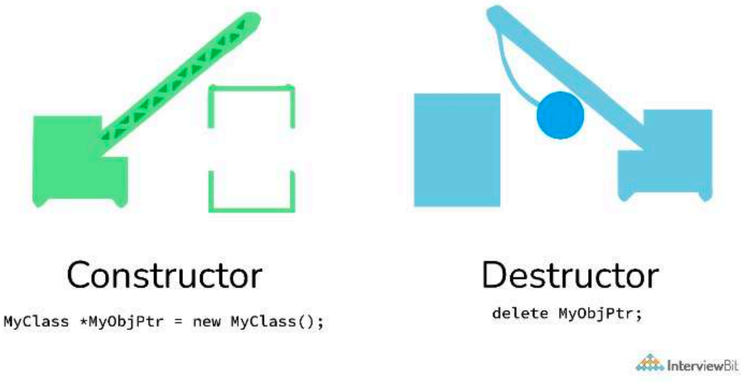
directly in this case, using the class name.

28. What is a constructor?

Constructors are special methods whose name is the same as the class name. The constructors serve the special purpose of initializing the objects.
For example, suppose there is a class with the name “MyClass”, then when you instantiate this class, you pass the syntax:

```
MyClass myClassObject = new MyClass();
```

Now here, the method called after “new” keyword - MyClass(), is the constructor of this class. This will help to instantiate the member data and methods and assign them to the object myClassObject.



29. What are the various types of constructors in C++?

The most common classification of constructors includes:

Default constructor: The default constructor is the constructor which doesn’t take any argument. It has no parameters.

```
class ABC
{
  int x;

  ABC()
  {
    x = 0;
  }
}
```

Parameterized constructor: The constructors that take some arguments are known as parameterized constructors.

```
class ABC
{
  int x;

  ABC(int y)
  {
    x = y;
  }
}
```

Copy constructor: A copy constructor is a member function that initializes an object using another object of the same class.

```
class ABC
{
  int x;

  ABC(int y)
  {
    x = y;
```

```
    x = 0;
  }
}
```

Parameterized constructor: The constructors that take some arguments are known as parameterized constructors.

```
class ABC
{
  int x;

  ABC(int y)
  {
    x = y;
  }
}
```

Copy constructor: A copy constructor is a member function that initializes an object using another object of the same class.

```
class ABC
{
  int x;

  ABC(int y)
  {
    x = y;
  }
  // Copy constructor
  ABC(ABC abc)
  {
    x = abc.x;
  }
}
```

30. What is a copy constructor?

Copy Constructor is a type of constructor, whose purpose is to copy an object to another. What it means is that a copy constructor will clone an object and its values, into another object, is provided that both the objects are of the same class.

31. What is a destructor?

Contrary to constructors, which initialize objects and specify space for them, Destructors are also special methods. But destructors free up the resources and memory occupied by an object. Destructors are automatically called when an object is being destroyed.

32. Are class and structure the same? If not, what's the difference between a class and a structure?

No, class and structure are not the same. Though they appear to be similar, they have differences that make them apart. For example, the structure is saved in the stack memory, whereas the class is saved in the heap memory. Also, Data Abstraction cannot be achieved with the help of structure, but with class, Abstraction is majorly used.

OOPs Coding Problems

1. What is the output of the below code?

```
#include<iostream>

using namespace std;
class BaseClass1 {
public:
   BaseClass1()
   { cout << " BaseClass1 constructor called" << endl;  }
};

class BaseClass2 {
public:
```

whereas the class is saved in the heap memory. Also, Data Abstraction cannot be achieved with the help of structure, but with class, Abstraction is majorly used.

OOPs Coding Problems

1. What is the output of the below code?

```
#include<iostream>

using namespace std;
class BaseClass1 {
public:
   BaseClass1()
   { cout << " BaseClass1 constructor called" << endl;  }
};

class BaseClass2 {
public:
   BaseClass2()
   { cout << "BaseClass2 constructor called" << endl;  }
};

class DerivedClass: public BaseClass1, public BaseClass2 {
  public:
   DerivedClass()
   {  cout << "DerivedClass constructor called" << endl;  }
};

int main()
{
  DerivedClass derived_class;
  return 0;
}
```

Output:

```
BaseClass1 constructor called
BaseClass2 constructor called
DerivedClass constructor called
```

Reason:

The above program demonstrates Multiple inheritances. So when the Derived class's constructor is called, it automatically calls the Base class's constructors from left to right order of inheritance.

2. What will be the output of the below code?

```
class Scaler
{
  static int i;

  static
  {
     System.out.println("a");

     i = 100;
  }
}

public class StaticBlock
{
  static
  {
     System.out.println("b");
  }

  public static void main(String[] args)
  {
     System.out.println("c");

     System.out.println(Scaler.i);
  }
}
```

constructor is called, it automatically calls the Base class's constructors from left to right order of inheritance.

2. What will be the output of the below code?

```
class Scaler
{
  static int i;

  static
  {
    System.out.println("a");

    i = 100;
  }
}

public class StaticBlock
{
  static
  {
    System.out.println("b");
  }

  public static void main(String[] args)
  {
    System.out.println("c");

    System.out.println(Scaler.i);
  }
}
```

Output:

```
b
c
a
100
```

Reason:

Firstly the static block inside the main-method calling class will be implemented. Hence 'b' will be printed first. Then the main method is called, and now the sequence is kept as expected.

3. Predict the output?

```
#include<iostream>
using namespace std;

class ClassA {
public:
  ClassA(int ii = 0) : i(ii) {}
  void show() { cout << "i = " << i << endl;}
private:
  int i;
};

class ClassB {
public:
  ClassB(int xx) : x(xx) {}
  operator ClassA() const { return ClassA(x); }
private:
  int x;
};

void g(ClassA a)
{  a.show(); }

int main() {
 ClassB b(10);
 g(b);
 g(20);
 getchar();
 return 0;
}
```

Firstly the static block inside the main-method calling class will be implemented. Hence 'b' will be printed first. Then the main method is called, and now the sequence is kept as expected.

3. Predict the output?

```
#include<iostream>
using namespace std;

class ClassA {
public:
  ClassA(int ii = 0) : i(ii) {}
  void show() { cout << "i = " << i << endl;}
private:
  int i;
};

class ClassB {
public:
  ClassB(int xx) : x(xx) {}
  operator ClassA() const { return ClassA(x); }
private:
  int x;
};

void g(ClassA a)
{  a.show(); }

int main() {
 ClassB b(10);
 g(b);
 g(20);
 getchar();
 return 0;
}
```

Output:

```
i = 10
i = 20
```

Reason:

ClassA contains a conversion constructor. Due to this, the objects of ClassA can have integer values. So the statement g(20) works. Also, ClassB has a conversion operator overloaded. So the statement g(b) also works.

4. What will be the output in below code?

```
public class Demo{
  public static void main(String[] arr){
      System.out.println("Main1");
  }
  public static void main(String arr){
      System.out.println("Main2");
  }
}
```

Output:

```
Main1
```

Reason:

Here the main() method is overloaded. But JVM only understands the main method which has a String[] argument in its definition. Hence Main1 is printed and the overloaded main method is ignored.

5. Predict the output?

```
#include<iostream>
using namespace std;

class BaseClass{
```

ignored.

5. Predict the output?

```
#include<iostream>
using namespace std;

class BaseClass{
  int arr[10];
};

class DerivedBaseClass1: public BaseClass { };

class DerivedBaseClass2: public BaseClass { };

class DerivedClass: public DerivedBaseClass1, public DerivedBaseClass2{};

int main(void)
{
 cout<<sizeof(DerivedClass);
 return 0;
}
```

Output:

```
If the size of the integer is 4 bytes, then the output will be 80.
```

Reason:

Since DerivedBaseClass1 and DerivedBaseClass2 both inherit from class BaseClass, DerivedClass contains two copies of BaseClass. Hence it results in wastage of space and a large size output. It can be reduced with the help of a virtual base class.

6. What is the output of the below program?

```
#include<iostream>

using namespace std;
class A {
public:
  void print()
  { cout <<" Inside A::"; }
};

class B : public A {
public:
  void print()
  { cout <<" Inside B"; }
};

class C: public B {
};

int main(void)
{
 C c;

 c.print();
 return 0;
}
```

Output:

```
Inside B
```

Reason:

The above program implements a Multi-level hierarchy. So the program is linearly searched up until a matching function is found. Here, it is present in both classes A and B. So class B's print() method is called.

Useful Resource

programming go through the questions and ace your upcoming interviews.

1. What is Object Oriented Programming (OOPs)?

Object Oriented Programming is a programming paradigm where the complete software operates as a bunch of objects talking to each other. An object is a collection of data and the methods which operate on that data.

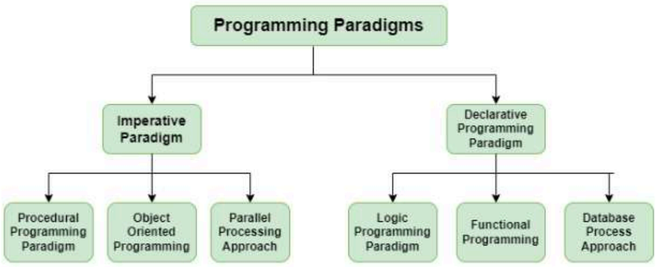
2. Why OOPs?

The main advantage of OOP is better manageable code that covers the following:

1. The overall understanding of the software is increased as the distance between the language spoken by developers and that spoken by users.
2. Object orientation eases maintenance by the use of encapsulation. One can easily change the underlying representation by keeping the methods the same.
3. The OOPs paradigm is mainly useful for relatively big software.

3. What other paradigms of programming exist besides OOPs?

The programming paradigm is referred to the technique or approach of writing a program. The programming paradigms can be classified into the following types:



1. Imperative Programming Paradigm

It is a programming paradigm that works by changing the program state through assignment statements. The main focus in this paradigm is on how to achieve the goal. The following programming paradigms come under this category:

1. **Procedural Programming Paradigm**: This programming paradigm is based on the procedure call concept. Procedures, also known as routines or functions are the basic building blocks of a program in this paradigm.
2. **Object-Oriented Programming or OOP**: In this paradigm, we visualize every entity as an object and try to structure the program based on the state and behavior of that object.
3. **Parallel Programming**: The parallel programming paradigm is the processing of instructions by dividing them into multiple smaller parts and executing them concurrently.

2. Declarative Programming Paradigm

Declarative programming focuses on what is to be executed rather than how it should be executed. In this paradigm, we express the logic of a computation without considering its control flow. The declarative paradigm can be further classified into:

1. **Logical Programming Paradigm**: It is based on formal logic where the program statements express the facts and rules about the problem in the logical form.
2. **Functional Programming Paradigm**: Programs are created by applying and composing functions in this paradigm.
3. **Database Programming Paradigm**: To manage data and information organized as fields, records, and files, database programming models are utilized.

4. What is the difference between Structured Programming and Object-Oriented Programming?

Structured Programming is a technique that is considered a precursor to OOP and usually consists of well-structured and separated modules. It is a subset of procedural programming. The difference between OOPs and Structured Programming is as follows:

Object-Oriented Programming	Structural Programming
Programming that is object-oriented is built	We logical structure is provided by structural

3. **Database Programming Paradigm**: To manage data and information organized as fields, records, and files, database programming models are utilized.

4. What is the difference between Structured Programming and Object-Oriented Programming?

Structured Programming is a technique that is considered a precursor to OOP and usually consists of well-structured and separated modules. It is a subset of procedural programming. The difference between OOPs and Structured Programming is as follows:

Object-Oriented Programming	Structural Programming
Programming that is object-oriented is built on objects having a state and behavior.	A program's logical structure is provided by structural programming, which divides programs into their corresponding functions.
It follows a bottom-to-top approach.	It follows a Top-to-Down approach.
Restricts the open flow of data to authorized parts only providing better data security.	No restriction to the flow of data. Anyone can access the data.
Enhanced code reusability due to the concepts of polymorphism and inheritance.	Code reusability is achieved by using functions and loops.
Methods work dynamically, making calls based on object behavior and the need of the code at runtime.	Functions are called sequentially, and code lines are processed step by step.
Modifying and updating the code is easier.	Modifying the code is difficult as compared to OOPs.
Data is given more importance in OOPs.	Code is given more importance.

5. What are some commonly used Object-Oriented Programming Languages?

OOPs paradigm is one of the most popular programming paradigms. It is widely used in many popular programming languages such as:

- C++
- Java
- Python
- JavaScript
- C#
- Ruby

6. What are the advantages and disadvantages of OOPs?

Advantages of OOPs	Disadvantages of OOPs
OOPs provides enhanced code reusability.	The programmer should be well-skilled and should have excellent thinking in terms of objects as everything is treated as an object in OOPs.
The code is easier to maintain and update.	Proper planning is required because OOPs is a little bit tricky.
It provides better data security by restricting data access and avoiding unnecessary exposure.	OOPs concept is not suitable for all kinds of problems.
Fast to implement and easy to redesign resulting in minimizing the complexity of an overall program.	The length of the programs is much larger in comparison to the procedural approach.

7. What is a Class?

A **class** is a building block of Object-Oriented Programs. It is a user-defined data type that contains the data members and member functions that operate on the data members. It is like a blueprint or template of objects having common properties and methods.

8. What is an Object?

An **object** is an instance of a class. Data members and methods of a class cannot be used directly. We need to create an object for instance of the class to use them. In simple terms, they are the

resulting in minimizing the complexity of an overall program.	The length of the programs is much larger in comparison to the procedural approach.

7. What is a Class?

A **class** is a building block of Object-Oriented Programs. It is a user-defined data type that contains the data members and member functions that operate on the data members. It is like a blueprint or template of objects having common properties and methods.

8. What is an Object?

An **object** is an instance of a class. Data members and methods of a class cannot be used directly. We need to create an object (or instance) of the class to use them. In simple terms, they are the actual world entities that have a state and behaviour.

C++ Java Python C#

```cpp
#include <iostream>
using namespace std;

// defining class
class Student {
public:
    string name;
};

int main()
{

    // creating object
    Student student1;
    // assigning member some value
    student1.name = "Rahul";

    cout << "student1.name: " << student1.name;

    return 0;
}
```

Output

```
student1.name: Rahul
```

9. What are the main features of OOPs?

The main feature of the OOPs, also known as 4 pillars or basic principles of OOPs are as follows:

1. Encapsulation
2. Data Abstraction
3. Polymorphism
4. Inheritance

OOPs Main Features

10. What is Encapsulation?

Encapsulation is the binding of data and methods that manipulate them into a single unit such that the sensitive data is hidden from the users
It is implemented as the processes mentioned below:

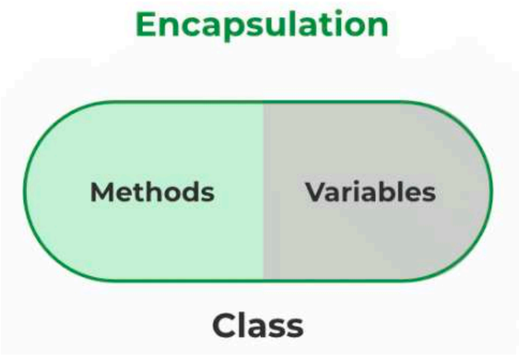
1. **Data hiding:** A language feature to restrict access to members of an object. For example, private and protected members in C++.
2. **Bundling of data and methods together:** Data and methods that operate on that data are bundled together. For example, the data members and member methods that operate on them are wrapped into a single unit known as a class.

OOPs Main Features

10. What is Encapsulation?

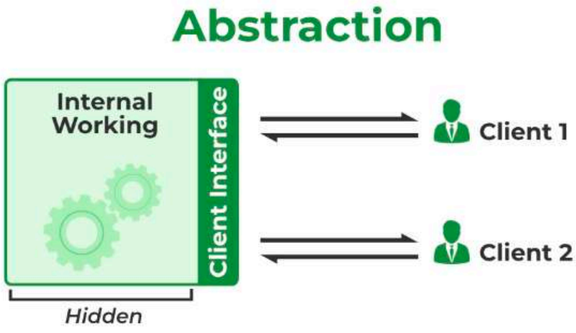
Encapsulation is the binding of data and methods that manipulate them into a single unit such that the sensitive data is hidden from the users
It is implemented as the processes mentioned below:

1. **Data hiding:** A language feature to restrict access to members of an object. For example, private and protected members in C++.
2. **Bundling of data and methods together:** Data and methods that operate on that data are bundled together. For example, the data members and member methods that operate on them are wrapped into a single unit known as a class.



11. What is Abstraction?

Abstraction is similar to data encapsulation and is very important in OOP. It means showing only the necessary information and hiding the other irrelevant information from the user. Abstraction is implemented using classes and interfaces.



12. What is Inheritance? What is its purpose?

The idea of inheritance is simple, a class is derived from another class and uses data and implementation of that other class. The class which is derived is called child or derived or subclass and the class from which the child class is derived is called parent or base or superclass.

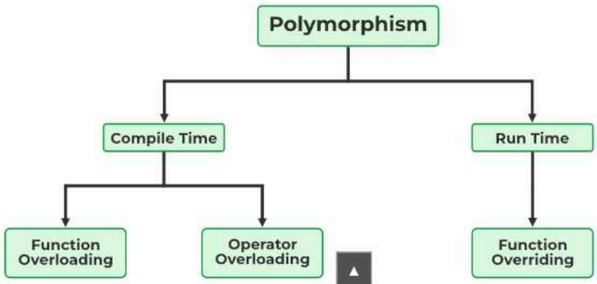
The main purpose of Inheritance is to increase code reusability. It is also used to achieve Runtime Polymorphism.

13. What is Polymorphism? and types of Polymorphism?

The word "**Polymorphism**" means having many forms. It is the property of some code to behave differently for different contexts. For example, in C++ language, we can define multiple functions having the same name but different working depending on the context.

Polymorphism can be classified into two types based on the time when the call to the object or function is resolved. They are as follows:

- Compile Time Polymorphism
- Runtime Polymorphism



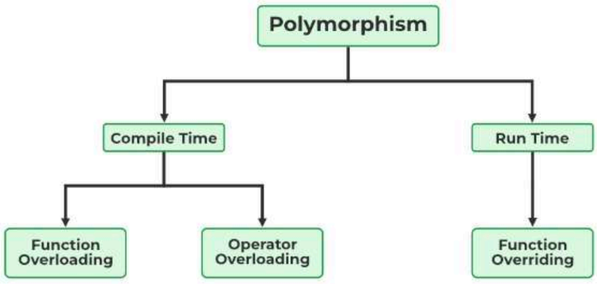
Polymorphism.

13. What is Polymorphism? and types of Polymorphism?

The word "**Polymorphism**" means having many forms. It is the property of some code to behave differently for different contexts. For example, in C++ language, we can define multiple functions having the same name but different working depending on the context.

Polymorphism can be classified into two types based on the time when the call to the object or function is resolved. They are as follows:

- Compile Time Polymorphism
- Runtime Polymorphism



A) Compile-Time Polymorphism

Compile time polymorphism, also known as static polymorphism or early binding is the type of polymorphism where the binding of the call to its code is done at the compile time. Method overloading or operator overloading are examples of compile-time polymorphism.

B) Runtime Polymorphism

Also known as dynamic polymorphism or late binding, runtime polymorphism is the type of polymorphism where the actual implementation of the function is determined during the runtime or execution. Function overriding is an example of this method.

14. What are access specifiers? What is their significance in OOPs?

Access specifiers are special types of keywords that are used to specify or control the accessibility of entities like classes, methods, and so on. **Private**, **Public**, and **Protected** are examples of access specifiers or access modifiers.
The key components of OOPs, encapsulation and data hiding, are largely achieved because of these access specifiers.

15. What is the difference between overloading and overriding?

A compile-time polymorphism feature called **overloading** allows an entity to have numerous implementations of the same name. Method overloading and operator overloading are two examples.

Overriding is a form of runtime polymorphism where an entity with the same name but a different implementation is executed. It is implemented with the help of virtual functions.

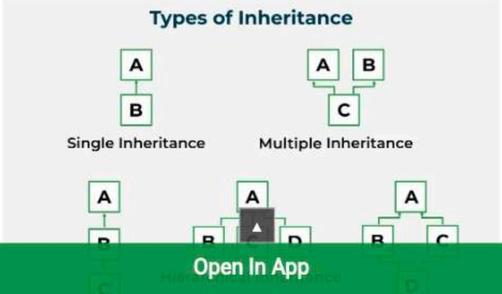
16. Are there any limitations on Inheritance?

Yes, there are more challenges when you have more authority. Although inheritance is a very strong OOPs feature, it also has significant drawbacks.

- As it must pass through several classes to be implemented, inheritance takes longer to process.
- The base class and the child class, which are both engaged in inheritance, are also closely related to one another (called tightly coupled). Therefore, if changes need to be made, they may need to be made in both classes at the same time.
- Implementing inheritance might be difficult as well. Therefore, if not implemented correctly, this could result in unforeseen mistakes or inaccurate outputs.

17. What different types of Inheritance are there?

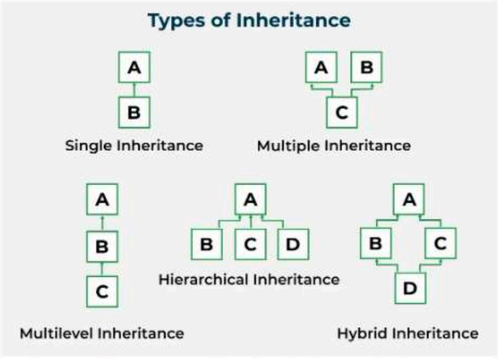
Inheritance can be classified into 5 types which are as follows:



need to be made in both classes at the same time.
- Implementing inheritance might be difficult as well. Therefore, if not implemented correctly, this could result in unforeseen mistakes or inaccurate outputs.

17. What different types of Inheritance are there?

Inheritance can be classified into 5 types which are as follows:



1. **Single Inheritance:** Child class derived directly from the base class
2. **Multiple Inheritance:** Child class derived from multiple base classes.
3. **Multilevel Inheritance:** Child class derived from the class which is also derived from another base class.
4. **Hierarchical Inheritance:** Multiple child classes derived from a single base class.
5. **Hybrid Inheritance:** Inheritance consisting of multiple inheritance types of the above specified.

> ***Note:*** *Type of inheritance supported is dependent on the language. For example, Java does not support multiple inheritance.*

18. What is an interface?

A unique class type known as an interface contains methods but not their definitions. Inside an interface, only method declaration is permitted. You cannot make objects using an interface. Instead, you must put that interface into use and specify the procedures for doing so.

19. How is an abstract class different from an interface?

Both abstract classes and interfaces are special types of classes that just include the declaration of the methods, not their implementation. An abstract class is completely distinct from an interface, though. Following are some major differences between an abstract class and an interface.

Abstract Class	Interface
A class that is abstract can have both abstract and non-abstract methods.	An interface can only have abstract methods.
An abstract class can have final, non-final, static and non-static variables.	The interface has only static and final variables.
Abstract class doesn't support multiple inheritance	An interface supports multiple inheritance.

20. How much memory does a class occupy?

Classes do not use memory. They merely serve as a template from which items are made. Now, objects actually initialize the class members and methods when they are created, using memory in the process.

21. Is it always necessary to create objects from class?

No. If the base class includes non-static methods, an object must be constructed. But no objects need to be generated if the class includes static methods. In this instance, you can use the class name to directly call those static methods.

22. What is the difference between a structure and a class in C++?

The structure is also a user-defined datatype in C++ similar to the class with the following differences:

- The major difference between a structure and a class is that in a structure, the members are set to public by default while in a class, members are private by default.
- The other difference is that we use **struct** for declaring structure and **class** for declaring a class in C++.

name to directly call those static methods.

22. What is the difference between a structure and a class in C++?

The structure is also a user-defined datatype in C++ similar to the class with the following differences:

- The major difference between a structure and a class is that in a structure, the members are set to public by default while in a class, members are private by default.
- The other difference is that we use **struct** for declaring structure and **class** for declaring a class in C++.

23. What is Constructor?

A constructor is a block of code that initializes the newly created object. A constructor resembles an instance method but it's not a method as it doesn't have a return type. It generally is the method having the same name as the class but in some languages, it might differ. For example:

In python, a constructor is named **__init__.**

In C++ and Java, the constructor is named the same as the class name.

Example:

C++ Java Python C#

```
class base {
    public:
        base() { cout << "This is a constructor"; }
}
```

24. What are the various types of constructors in C++?

The most common classification of constructors includes:

1. **Default Constructor**
2. **Non-Parameterized Constructor**
3. **Parameterized Constructor**
4. **Copy Constructor**

1. Default Constructor

The default constructor is a constructor that doesn't take any arguments. It is a non-parameterized constructor that is automatically defined by the compiler when no explicit constructor definition is provided.

It initializes the data members to their default values.

2. Non-Parameterized Constructor

It is a user-defined constructor having no arguments or parameters.

Example:

C++ Java Python C#

```
class base {
public:
    base(){
        cout << "This is a non-parameterized contructor";
    }
}
```

3. Parameterized Constructor

The constructors that take some arguments are known as parameterized constructors.

Example:

C++ Java Python C#

```
class base {
public:
    int base;
```

24. What are the various types of constructors in C++?

The most common classification of constructors includes:

1. **Default Constructor**
2. **Non-Parameterized Constructor**
3. **Parameterized Constructor**
4. **Copy Constructor**

1. Default Constructor

The default constructor is a constructor that doesn't take any arguments. It is a non-parameterized constructor that is automatically defined by the compiler when no explicit constructor definition is provided.

It initializes the data members to their default values.

2. Non-Parameterized Constructor

It is a user-defined constructor having no arguments or parameters.

Example:

C++ Java Python C#

```
class base {
public:
    base(){
        cout << "This is a non-parameterized contructor";
    }
}
```

3. Parameterized Constructor

The constructors that take some arguments are known as parameterized constructors.

Example:

C++ Java Python C#

```
class base {
public:
    int base;
    base(int var) {
        cout << "Constructor with argument: " << var;
    }
};
```

4. Copy Constructor

A copy constructor is a member function that initializes an object using another object of the same class.

Example:

C++ Java Python C#

```
class base {
    int a, b;

    // copy constructor
    base(base& obj) {
        a = obj.a;
        b = obj.b;
    }
}
```

In Python, we do not have built-in copy constructors like Java and C++ but we can make a workaround using different methods.

25. What is a destructor?

A destructor is a method that is automatically called when the object goes out of scope or destroyed.

In C++, the destructor name is also the same as the class name but with the (~) **tilde symbol** as the prefix.

In Python, the destructor is named **__del__**.

Example:

C++ Java Python C#

```
class base {
public:
    ~base() { cout << "This is a destructor"; }
}
```

In Python, we do not have built-in copy constructors like Java and C++ but we can make a workaround using different methods.

25. What is a destructor?

A destructor is a method that is automatically called when the object goes out of scope or destroyed.

In C++, the destructor name is also the same as the class name but with the (~) **tilde symbol** as the prefix.

In Python, the destructor is named **__del__**.

Example:

C++ Java Python C#

```
class base {
public:
    ~base() { cout << "This is a destructor"; }
}
```

In Java, the garbage collector automatically deletes the useless objects so there is no concept of destructor in Java. We could have used finalize() method as a workaround for the java destructor, but it is also deprecated since Java 9.

26. Can we overload the constructor in a class?

Yes We can overload the constructor in a class in Java. Constructor Overloading is done when we want constructor with different constructor with different parameter (Number and Type).

27. Can we overload the destructor in a class?

No, a destructor cannot be overloaded in a class. There can only be one destructor present in a class.

28. What are friend functions and friend classes?

Friend Functions: A friend function is a special function that is allowed to access private and protected data of a class, even though it's not a member of the class.

Friend Class: A friend class is a class that can access the private and protected members of another class. It's like allowing a trusted friend or a group of friends to see and change your personal information, which others cannot.

29. What is the virtual function and pure virtual function?

A virtual function is a function that is used to override a method of the parent class in the derived class. It is used to provide abstraction in a class.

- In C++ and C#, a virtual function is declared using the virtual keyword,
- In Java, every public, non-static, and non-final method is a virtual function.
- Python methods are always virtual.

Example:

C++ Java Python C#

```
class base {
    virtual void print() {
        cout << "This is a virtual function";
    }
}
```

A pure virtual function, also known as an abstract function, is a member function that doesn't contain any statements. This function is defined in the derived class if needed.

Example:

C++ Java C#

```
class base {
    virtual void pureVirFunc() = 0;
}
```

In Python, we achieve this using @abstractmethod from the ABC (Abstract Base Class) module.

30. What is an abstract class?

In general terms, an abstract class is a class that is intended to be used for inheritance. It cannot be instantiated. An abstract class can consist of both abstract and non-abstract methods.

- In C++, an abstract class is a class that contains one pure virtual function.

- Python methods are always virtual.

Example:

C++ Java Python C#

```
class base {
    virtual void print() {
        cout << "This is a virtual function";
    }
}
```

A pure virtual function, also known as an abstract function, is a member function that doesn't contain any statements. This function is defined in the derived class if needed.

Example:

C++ Java C#

```
class base {
    virtual void pureVirFunc() = 0;
}
```

In Python, we achieve this using @abstractmethod from the ABC (Abstract Base Class) module.

30. What is an abstract class?

In general terms, an abstract class is a class that is intended to be used for inheritance. It cannot be instantiated. An abstract class can consist of both abstract and non-abstract methods.

- In C++, an abstract class is a class that contains at least one pure virtual function.
- In Java and C#, an abstract class is declared with an **abstract** keyword.

Example:

C++ Java C#

```
class absClass {
public:
    virtual void pvFunc() = 0;
}
```

In Python, we use ABC (Abstract Base Class) module to create an abstract class.

Must Refer:

1. OOPs in C++
2. OOPs in Java
3. OOPs in Python
4. Classes and Objects in C++
5. Classes and Objects in Java
6. Classes and Objects in Python
7. Introduction to Programming Paradigms
8. Interface in Java
9. Abstract Class in Java
10. C++ Interview Questions

Basic OOPs Interview Questions for Freshers

1. What is the difference between OOP and SOP?

Object-Oriented Programming	Structural Programming
Object-Oriented Programming is a type of programming which is based on objects rather than just functions and procedures	Provides logical structure to a program where programs are divided functions
Bottom-up approach	Top-down approach
Provides data hiding	Does not provide data hiding
Can solve problems of any complexity	Can solve moderate problems
Code can be reused thereby reducing redundancy	Does not support code reusability

2. What is Object Oriented Programming?

Object-Oriented Programming(OOPs) is a type of programming that is based on objects rather than just functions and procedures. Individual objects are grouped into classes. OOPs implements real-world entities like inheritance, polymorphism, hiding, etc into programming. It also allows binding data and code together.

3. Why use OOPs?

- OOPs allows clarity in programming thereby allowing simplicity in solving complex problems
- Code can be reused through inheritance thereby reducing redundancy
- Data and code are bound together by encapsulation
- OOPs allows data hiding, therefore, private data is kept confidential
- Problems can be divided into different parts making it simple to solve
- The concept of polymorphism gives flexibility to the program by allowing the entities to have multiple forms

4. What are the main features of OOPs?

- Inheritance
- Encapsulation
- Polymorphism
- Data Abstraction

To know more about OOPs in JAVA, Python, and C++ you can go through the following blogs:

- **JAVA** OOPs Concepts
- **Python** OOPs Concepts
- **C++** OOPs Concepts

Classes and Objects OOP Interview Questions and Answers

5. What is an object?

An object is a real-world entity which is the basic unit of OOPs for example chair, cat, dog, etc. Different objects have different states or attributes, and behaviors.

6. What is a class?

A class is a prototype that consists of objects in different states and with different behaviors. It has a number of methods that are common the objects present within that class.

7. What is the difference between a class and a structure?

Class: User-defined blueprint from which objects are created. It consists of methods or set

It has a number of methods that are common the objects present within that class.

7. What is the difference between a class and a structure?

Class: User-defined blueprint from which objects are created. It consists of methods or set of instructions that are to be performed on the objects.

Structure: A structure is basically a user-defined collection of variables which are of different data types.

8. Can you call the base class method without creating an instance?

Yes, you can call the base class without instantiating it if:

- It is a static method
- The base class is inherited by some other subclass

9. What is the difference between a class and an object?

Object	Class
A real-world entity which is an instance of a class	A class is basically a template or a blueprint within which objects can be created
An object acts like a variable of the class	Binds methods and data together into a single unit
An object is a physical entity	A class is a logical entity
Objects take memory space when they are created	A class does not take memory space when created
Objects can be declared as and when required	Classes are declared just once

To know more about objects and classes in JAVA, Python, and C++ you can go through the following blogs:

- Objects in **Java**
- Class in **Java**
- Objects and classes in **Python**
- Objects in **C++**

OOPs Interview Questions – Inheritance

10. What is inheritance?

Inheritance is a feature of OOPs which allows classes inherit common properties from other classes. For example, if there is a class such as 'vehicle', other classes like 'car', 'bike', etc can inherit common properties from the vehicle class. This property helps you get rid of redundant code thereby reducing the overall size of the code.

11. What are the different types of inheritance?

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Hybrid inheritance

12. What is the difference between multiple and multilevel inheritance?

Multiple Inheritance	Multilevel Inheritance
Multiple inheritance comes into picture when a class inherits more than one base class	Multilevel inheritance means a class inherits from another class which itself is a subclass of some other base class
Example: A class defining a child inherits from two base classes Mother and Father	Example: A class describing a sports car will inherit from a base class Car which inturn inherits another class Vehicle

13. What is hybrid inheritance?

Hybrid inheritance is a combination of multiple and multi-level inherit

14. What is hierarchical inheritance?

Example: A class defining a child inherits from two base classes Mother and Father	Example: A class describing a sports car will inherit from a base class Car which inturn inherits another class Vehicle

13. What is hybrid inheritance?

Hybrid inheritance is a combination of multiple and multi-level inheritance.

14. What is hierarchical inheritance?

Hierarchical inheritance refers to inheritance where one base class has more than one subclasses. For example, the vehicle class can have 'car', 'bike', etc as its subclasses.

15. What are the limitations of inheritance?

- Increases the time and effort required to execute a program as it requires jumping back and forth between different classes
- The parent class and the child class get tightly coupled
- Any modifications to the program would require changes both in the parent as well as the child class
- Needs careful implementation else would lead to incorrect results

To know more about inheritance in Java and Python, read the below articles:

- Inheritance in Java
- Inheritance in Python

16. What is a superclass?

A superclass or base class is a class that acts as a parent to some other class or classes. For example, the Vehicle class is a superclass of class Car.

17. What is a subclass?

A class that inherits from another class is called the subclass. For example, the class Car is a subclass or a derived of Vehicle class.

Want to upskill yourself to get ahead in your Career? Check out this video

Top 10 Technologies to Learn in 2025 | Trending Technologies in 2025 | Edureka

Technologies to Learn helps you learn about new trending technology

OOP Interview Questions – Polymor

OOP Interview Questions – Polymorphism

18. What is polymorphism?

Polymorphism refers to the ability to exist in multiple forms. Multiple definitions can be given to a single interface. For example, if you have a class named Vehicle, it can have a method named speed but you cannot define it because different vehicles have different speed. This method will be defined in the subclasses with different definitions for different vehicles.

19. What is static polymorphism?

Static polymorphism (static binding) is a kind of polymorphism that occurs at compile time. An example of compile-time polymorphism is method overloading.

20. What is dynamic polymorphism?

Runtime polymorphism or dynamic polymorphism (dynamic binding) is a type of polymorphism which is resolved during runtime. An example of runtime polymorphism is method overriding.

21. What is method overloading?

Method overloading is a feature of OOPs which makes it possible to give the same name to more than one methods within a class if the arguments passed differ.

22. What is method overriding?

Method overriding is a feature of OOPs by which the child class or the subclass can redefine methods present in the base class or parent class. Here, the method that is overridden has the same name as well as the signature meaning the arguments passed and the return type.

23. What is operator overloading?

Operator overloading refers to implementing operators using user-defined types based on the arguments passed along with it.

24. Differentiate between overloading and overriding.

Overloading	Overriding
Two or more methods having the same name but different parameters or signature	Child class redefining methods present in the base class with the same parameters/ signature
Resolved during compile-time	Resolved during runtime

To know more about polymorphism in Java and Python, read the below articles:

- Polymorphism in Java
- Polymorphism in Python

OOPs Interview Questions – Encapsulation

25. What is encapsulation?

Encapsulation refers to binding the data and the code that works on that together in a single unit. For example, a class. Encapsulation also allows data-hiding as the data specified in one class is hidden from other classes.

26. What are 'access specifiers'?

Access specifiers or access modifiers are keywords that determine the accessibility of methods, classes, etc in OOPs. These access specifiers allow the implementation of encapsulation. The most common access specifiers are public, private and protected. However, there are a few more which are specific to the programming languages.

27. What is the difference between public, private and protected access modifiers?

Name	Accessibility from own class	Accessibility from derived class	Accessibility from world
Public	Yes	Yes	Yes
Private	Yes	No	

However, there are a few more which are specific to the programming languages.

27. What is the difference between public, private and protected access modifiers?

Name	Accessibility from own class	Accessibility from derived class	Accessibility from world
Public	Yes	Yes	Yes
Private	Yes	No	No
Protected	Yes	Yes	No

To know more about encapsulation read along:

- Encapsulation in Java
- Encapsulation in C++
- Encapsulation in Python

Data abstraction

28. What is data abstraction?

Data abstraction is a very important feature of OOPs that allows displaying only the important information and hiding the implementation details. For example, while riding a bike, you know that if you raise the accelerator, the speed will increase, but you don't know how it actually happens. This is data abstraction as the implementation details are hidden from the rider.

29. How to achieve data abstraction?

Data abstraction can be achieved through:

- Abstract class
- Abstract method

30. What is an abstract class?

An abstract class is a class that consists of abstract methods. These methods are basically declared but not defined. If these methods are to be used in some subclass, they need to be exclusively defined in the subclass.

31. Can you create an instance of an abstract class?

No. Instances of an abstract class cannot be created because it does not have a complete implementation. However, instances of subclass inheriting the abstract class can be created.

32. What is an interface?

It is a concept of OOPs that allows you to declare methods without defining them. Interfaces, unlike classes, are not blueprints because they do not contain detailed instructions or actions to be performed. Any class that implements an interface defines the methods of the interface.

33. Differentiate between data abstraction and encapsulation.

Data abstraction	Encaps
Solves the problem at the design level	Solves the problem at

actions to be performed. Any class that implements an interface defines the methods of the interface.

33. Differentiate between data abstraction and encapsulation.

Data abstraction	Encapsulation
Solves the problem at the design level	Solves the problem at the implementation level
Allows showing important aspects while hiding implementation details	Binds code and data together into a single unit and hides it from the world

To know more about data abstraction, below articles might help you:

- Abstraction in **Java**
- Abstraction in **Python**

Methods and Functions OOPs interview questions

34. What are virtual functions?

Virtual functions are functions that are present in the parent class and are overridden by the subclass. These functions are used to achieve runtime polymorphism.

35. What are pure virtual functions?

Pure virtual functions or abstract functions are functions that are only declared in the base class. This means that they do not contain any definition in the base class and need to be redefined in the subclass.

36. What is a constructor?

A constructor is a special type of method that has the same name as the class and is used to initialize objects of that class.

37. What is a destructor?

A destructor is a method that is automatically invoked when an object is destroyed. The destructor also recovers the heap space that was allocated to the destroyed object, closes the files and database connections of the object, etc.

38. Types of constructors

Types of constructors differ from language to language. However, all the possible constructors are:

- Default constructor
- Parameterized constructor
- Copy constructor
- Static constructor
- Private constructor

39. What is a copy constructor?

A copy constructor creates objects by copying variables from another object of the same class. The main aim of a copy constructor is to create a new object from an existing one.

40. What is the use of 'finalize'?

Finalize as an object method used to free up unmanaged resources and cleanup before Garbage Collection(GC). It performs memory management tasks.

41. What is Garbage Collection(GC)?

GC is an implementation of automatic memory management. The Garbage collector frees up space occupied by objects that are no longer in existence.

42. Differentiate between a class and a method.

Class	Method
A class is basically a template that binds the code and data together into a single unit. Classes consist of methods, variables, etc	Callable set of instructions also called a procedure or function that are to be performed on the given data

43. Differentiate between an abstract class and an interface?

Basis for comparison	Abstract Class

GC is an implementation of automatic memory management. The Garbage collector frees up space occupied by objects that are no longer in existence.

42. Differentiate between a class and a method.

Class	Method
A class is basically a template that binds the code and data together into a single unit. Classes consist of methods, variables, etc	Callable set of instructions also called a procedure or function that are to be performed on the given data

43. Differentiate between an abstract class and an interface?

Basis for comparison	Abstract Class	Interface
Methods	Can have abstract as well as other methods	Only abstract methods
Final Variables	May contain final and non-final variables	Variables declared are final by default
Accessibility of Data Members	Can be private, public, etc	Public by default
Implementation	Can provide the implementation of an interface	Cannot provide the implementation of an abstract class

44. What is a final variable?

A variable whose value does not change. It always refers to the same object by the property of non-transversity.

OOPs Interview Questions – Exception Handling

45. What is an exception?

An exception is a kind of notification that interrupts the normal execution of a program. Exceptions provide a pattern to the error and transfer the error to the exception handler to resolve it. The state of the program is saved as soon as an exception is raised.

46. What is exception handling?

Exception handling in Object-Oriented Programming is a very important concept that is used to manage errors. An exception handler allows errors to be thrown and caught and implements a centralized mechanism to resolve them.

47. What is the difference between an error and an exception?

Error	Exception
Errors are problems that should not be encountered by applications	Conditions that an application might try to catch

48. What is a try/ catch block?

A try/ catch block is used to handle exceptions. The try block defines a set of statements that may lead to an error. The catch block basically catches the exception.

49. What is a finally block?

A finally block consists of code that is used to execute important code

Error	Exception
Errors are problems that should not be encountered by applications	Conditions that an application might try to catch

48. What is a try/ catch block?

A try/ catch block is used to handle exceptions. The try block defines a set of statements that may lead to an error. The catch block basically catches the exception.

49. What is a finally block?

A finally block consists of code that is used to execute important code such as closing a connection, etc. This block executes when the try block exits. It also makes sure that finally block executes even in case some unexpected exception is encountered.

OOPs Interview Questions – Limitations of OOPs

50. What are the limitations of OOPs?

- Usually not suitable for small problems
- Requires intensive testing
- Takes more time to solve the problem
- Requires proper planning
- The programmer should think of solving a problem in terms of objects

Hope you are clear with all that has been shared with you in this tutorial. This brings us to the end of our article on OOP Interview Questions. ***Make sure you practice as much as possible and revert your experience.***

To get in-depth knowledge on this concept, you can check out the live ***Python Online training*** *and* ***Java Certification Training*** *with 24/7 support and lifetime access.*

Got a question for us? Please mention it in the comments section of this "OOPS Interview Questions" blog and we will get back to you as soon as possible.